

Seguridad de la Información

Práctica 2: Algoritmos de resumen y cifrado simétrico

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Sergio Acosta Quintana

Fecha: 13 de febrero de 2026

Índice general

1. Introducción y Configuración	2
1.1. Configuración del Entorno	2
2. Integridad y Hashing (Resúmenes)	4
2.1. Generación de Huellas Digitales	4
2.2. El Efecto Avalancha	5
2.3. Análisis de Formatos y Tamaños	6
3. Cifrado Simétrico	9
3.1. Proceso de Cifrado y Algoritmos	9
3.2. Análisis del Tamaño del Cifrado (Padding)	10
3.3. Derivación de Claves (KDF)	11
3.4. Descifrado Manual (Avanzado)	11
4. Análisis de Vulnerabilidad: Modo ECB	14
4.1. Manipulación de Imágenes (Netpbm)	14
4.2. El Problema de los Patrones Visuales	15
4.3. Ejecución del Ataque	15
5. Bibliografía	18

Capítulo 1

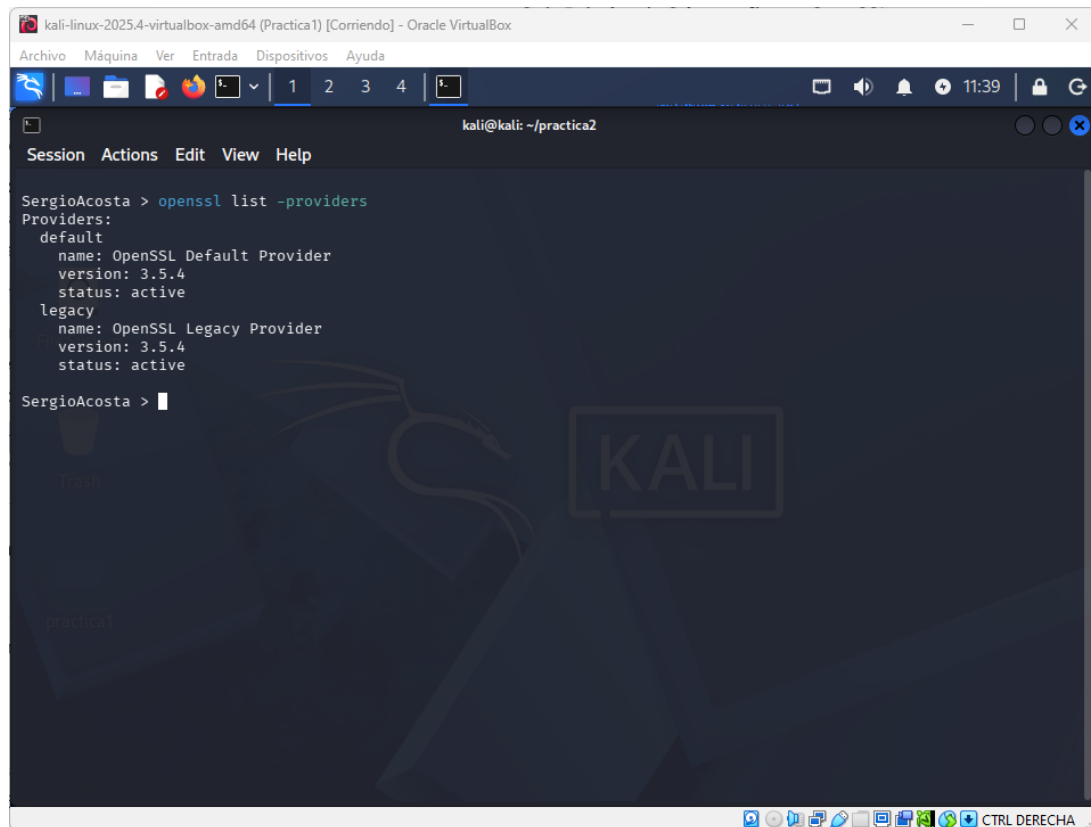
Introducción y Configuración

El presente informe documenta el desarrollo de la segunda práctica de laboratorio de la asignatura Seguridad de la Información. El objetivo principal ha sido familiarizarse con el uso de herramientas criptográficas en un entorno Linux, específicamente utilizando la suite **OpenSSL** sobre una distribución Kali Linux.

A lo largo de este documento, analizaremos el ciclo de vida de la protección de datos, abarcando desde la garantía de integridad mediante funciones hash, hasta la confidencialidad mediante cifrado simétrico, finalizando con una demostración práctica de las vulnerabilidades asociadas a una mala elección del modo de operación.

1.1. Configuración del Entorno

Antes de comenzar con las operaciones criptográficas, es fundamental verificar que el entorno de trabajo dispone de los módulos necesarios. En versiones recientes de OpenSSL (versión 3.0 en adelante), ciertos algoritmos considerados obsoletos o inseguros (como MD5, SHA1 o DES) se han movido a un proveedor de servicios denominado *Legacy*.



```
kali@kali: ~/practica2
Session Actions Edit View Help

SergioAcosta > openssl list -providers
Providers:
  default
    name: OpenSSL Default Provider
    version: 3.5.4
    status: active
  legacy
    name: OpenSSL Legacy Provider
    version: 3.5.4
    status: active

SergioAcosta > 
```

Figura 1.1: Verificación de proveedores activos en OpenSSL.

Como se aprecia en la captura de la Figura 1.1, mediante el comando `openssl list -providers`, confirmamos que tanto el proveedor `default` (para algoritmos modernos como AES y SHA2) como el proveedor `legacy` están activos. Esto garantiza que podremos realizar ejercicios académicos con algoritmos antiguos sin errores de ejecución.

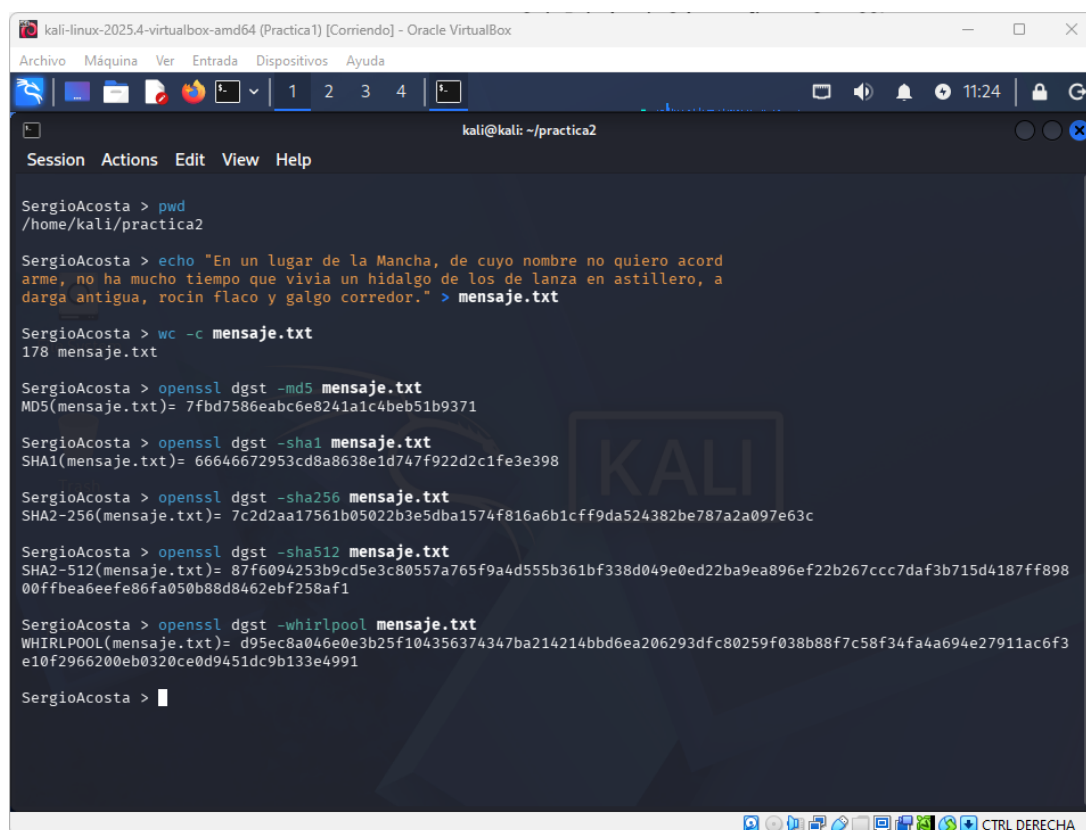
Capítulo 2

Integridad y Hashing (Resúmenes)

La integridad de la información es uno de los pilares de la seguridad (CIA: Confidentiality, Integrity, Availability). Para garantizarla, utilizamos funciones resumen o *hash*, que convierten una entrada de longitud arbitraria en una salida de longitud fija.

2.1. Generación de Huellas Digitales

En esta fase, hemos creado un archivo de prueba denominado `mensaje.txt` conteniendo la famosa frase inicial de "Don Quijote". El archivo resultante tiene un tamaño de 178 bytes. Posteriormente, hemos calculado su huella digital utilizando diversos algoritmos disponibles en OpenSSL.



```
kali@kali: ~/practica2

SergioAcosta > pwd
/home/kali/practica2

SergioAcosta > echo "En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivia un hidalgo de los de lanza en astillero, a darga antigua, rocin flaco y galgo corredor." > mensaje.txt

SergioAcosta > wc -c mensaje.txt
178 mensaje.txt

SergioAcosta > openssl dgst -md5 mensaje.txt
MD5(mensaje.txt)= 7fbd7586eabc6e8241a1c4beb51b9371

SergioAcosta > openssl dgst -sha1 mensaje.txt
SHA1(mensaje.txt)= 66646672953cd8a8638e1d747f922d2c1fe3e398

SergioAcosta > openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 7c2d2aa17561b05022b3e5dba1574f816a6b1cfff9da524382be787a2a097e63c

SergioAcosta > openssl dgst -sha512 mensaje.txt
SHA2-512(mensaje.txt)= 87f6094253b9cd5e3c80557a765f9a4d555b361bf338d049e0ed22ba9ea896ef22b267ccc7daf3b715d4187ff89800ffbea6eefe86fa050b88d8462ebf258af1

SergioAcosta > openssl dgst -whirlpool mensaje.txt
WHIRLPOOL(mensaje.txt)= d95ec8a046e0e3b25f104356374347ba214214bbd6ea206293dfc80259f038b88f7c58f34fa4a694e27911ac6f3e10f2966200eb0320ce0d9451dc9b133e4991

SergioAcosta > 
```

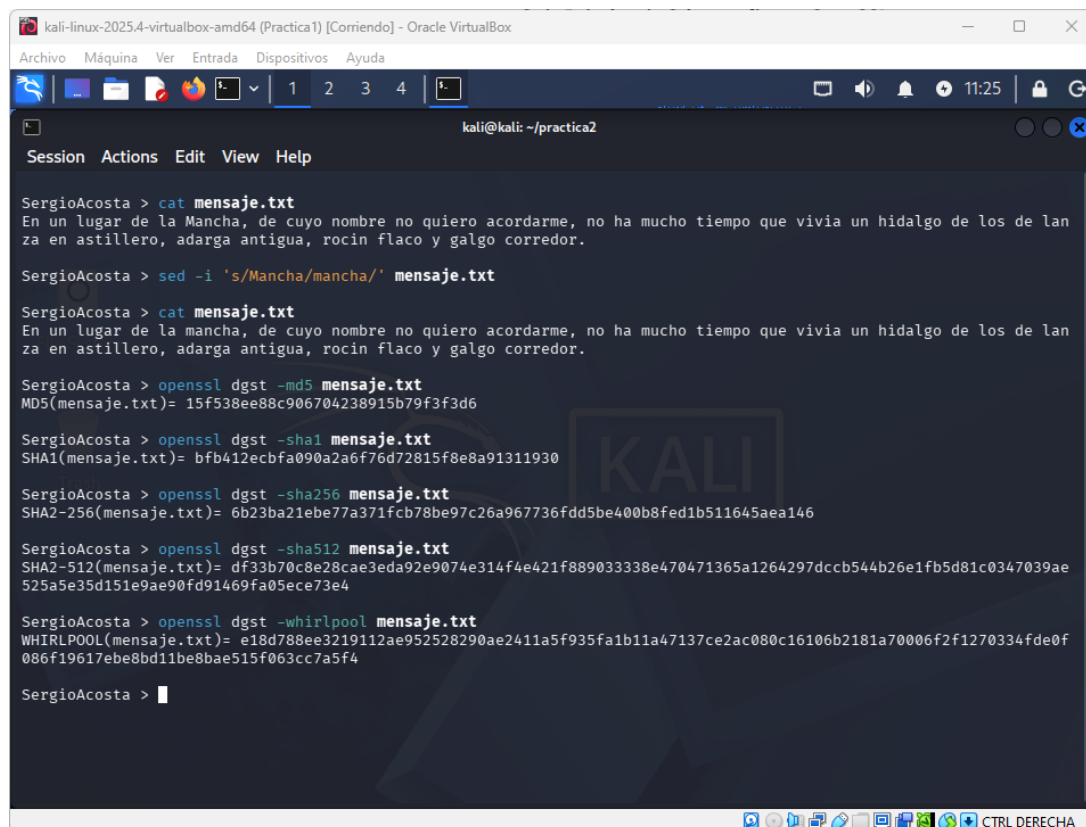
Figura 2.1: Cálculo de hashes con múltiples algoritmos.

Los algoritmos utilizados y sus características son:

- **MD5 (Message Digest 5):** Genera un hash de 128 bits. Actualmente se considera criptográficamente roto debido a la facilidad para encontrar colisiones.
- **SHA1 (Secure Hash Algorithm 1):** Genera 160 bits. Aunque fue un estándar durante años, Google demostró colisiones prácticas en 2017 (ataque SHAttered), por lo que ya no es válido para firmas digitales.
- **SHA256 y SHA512:** Pertenecen a la familia SHA-2. Son los estándares actuales recomendados por el NIST para la mayoría de aplicaciones de seguridad.
- **Whirlpool:** Un algoritmo menos común diseñado tras el estándar AES, que produce un resumen de 512 bits, ofreciendo una resistencia muy alta.

2.2. El Efecto Avalancha

Una propiedad esencial de una buena función hash criptográfica es el denominado "Efecto Avalancha". Este principio establece que un cambio minúsculo en la entrada (por ejemplo, cambiar un solo bit) debe provocar un cambio drástico e impredecible en la salida (idealmente, afectando al 50 % de los bits del hash).



```
kali@kali: ~/practica2

SergioAcosta > cat mensaje.txt
En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivia un hidalgo de los de lanza en astillero, adarga antigua, rocin flaco y galgo corredor.

SergioAcosta > sed -i 's/Mancha/mancha/' mensaje.txt

SergioAcosta > cat mensaje.txt
En un lugar de la mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivia un hidalgo de los de lanza en astillero, adarga antigua, rocin flaco y galgo corredor.

SergioAcosta > openssl dgst -md5 mensaje.txt
MD5(mensaje.txt)= 15f538ee88c906704238915b79f3f3d6

SergioAcosta > openssl dgst -sha1 mensaje.txt
SHA1(mensaje.txt)= bfb412ecbfa090a2a6f76d72815f8e8a91311930

SergioAcosta > openssl dgst -sha256 mensaje.txt
SHA2-256(mensaje.txt)= 6b23ba21ebe77a371fcb78be97c26a967736fdd5be400b8fed1b511645aea146

SergioAcosta > openssl dgst -sha512 mensaje.txt
SHA2-512(mensaje.txt)= df33b70c8e28cae3eda92e9074e314f4e421f889033338e470471365a1264297dcc544b26e1fb5d81c0347039ae525a5e35d151e9ae90fd91469fa05ece73e4

SergioAcosta > openssl dgst -whirlpool mensaje.txt
WHIRLPOOL(mensaje.txt)= e18d788ee3219112ae952528290ae2411a5f935fa1b11a47137ce2ac080c16106b2181a70006f2f1270334fde0f086f19617ebe8bd11be8bae515f063cc7a5f4

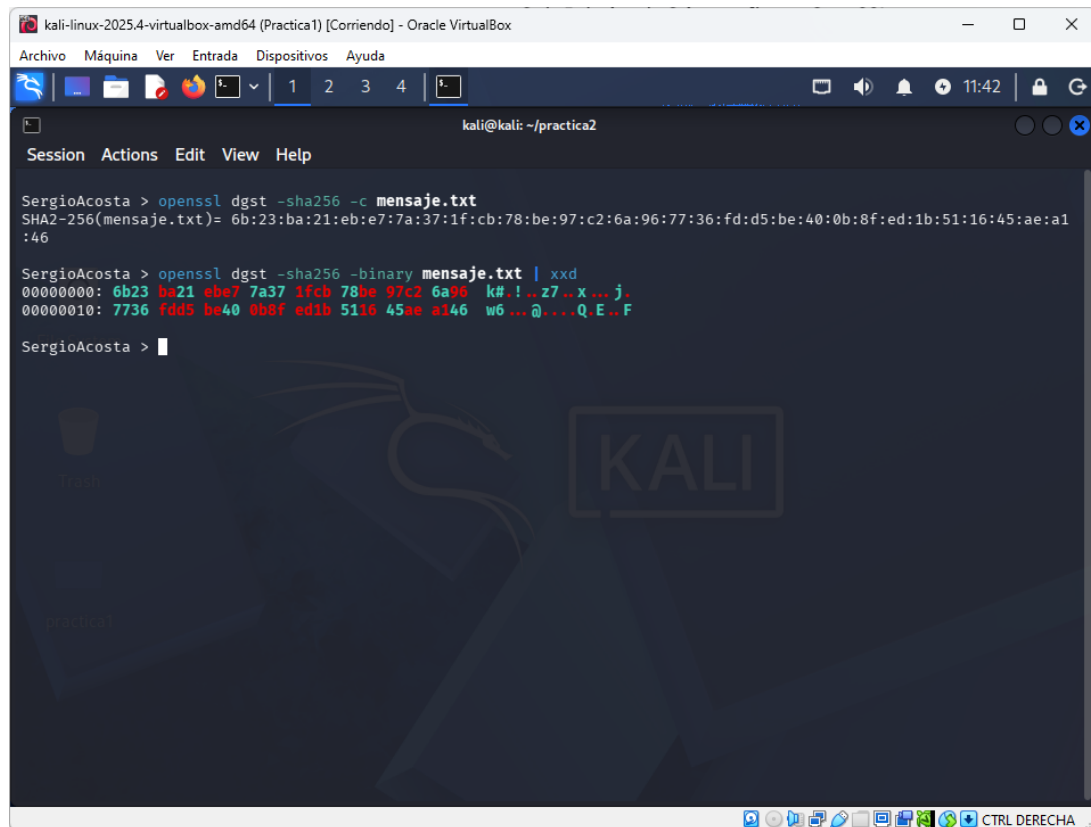
SergioAcosta > 
```

Figura 2.2: Demostración del efecto avalancha tras modificar un bit.

Para comprobarlo, utilizamos el comando `sed` para alterar una única letra en `mensaje.txt`: cambiamos la "M" mayúscula de "Mancha" por una "m" minúscula. Aunque semánticamente el texto es casi idéntico, a nivel binario hemos cambiado bits específicos. Al recalcular los hashes, observamos que las cadenas resultantes son **completamente diferentes** a las originales.

2.3. Análisis de Formatos y Tamaños

Es crucial diferenciar entre la representación del hash y su valor real. Por defecto, OpenSSL muestra el hash en formato hexadecimal (base 16) para que sea legible por humanos.



```
kali@kali: ~/practica2
Session Actions Edit View Help

SergioAcosta > openssl dgst -sha256 -c mensaje.txt
SHA2-256(mensaje.txt)= 6b:23:ba:21:eb:e7:7a:37:1f:cb:78:be:97:c2:6a:96:77:36:fd:d5:be:40:0b:8f:ed:1b:51:16:45:ae:a1:46

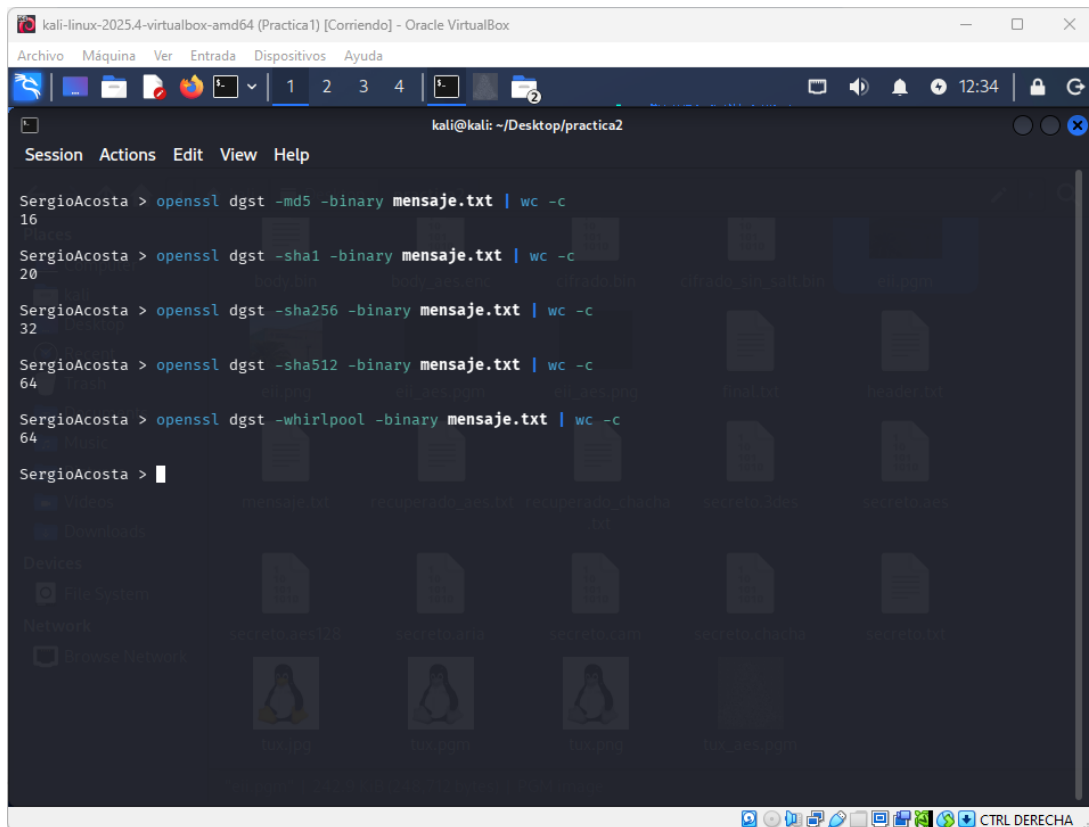
SergioAcosta > openssl dgst -sha256 -binary mensaje.txt | xxd
00000000: 6b23 ba21 ebe7 7a37 1fcb 78be 97c2 6a96  k#..z7..x...j.
00000010: 7736 fdd5 be40 0b8f ed1b 5116 45ae a146  w6...@...Q.E..F

SergioAcosta > 
```

Figura 2.3: Diferentes formatos de visualización de hashes.

En la Figura 2.4 hemos utilizado el flag `-binary` junto con `wc -c` para contar los bytes reales de salida. Los resultados confirman la teoría matemática de cada algoritmo:

- **MD5:** 128 bits / 8 = **16 bytes**.
- **SHA1:** 160 bits / 8 = **20 bytes**.
- **SHA256:** 256 bits / 8 = **32 bytes**.
- **SHA512:** 512 bits / 8 = **64 bytes**.



```
kali@kali: ~/Desktop/practica2
Session Actions Edit View Help

SergioAcosta > openssl dgst -md5 -binary mensaje.txt | wc -c
16

SergioAcosta > openssl dgst -sha1 -binary mensaje.txt | wc -c
20

SergioAcosta > openssl dgst -sha256 -binary mensaje.txt | wc -c
32

SergioAcosta > openssl dgst -sha512 -binary mensaje.txt | wc -c
64

SergioAcosta > openssl dgst -whirlpool -binary mensaje.txt | wc -c
64

SergioAcosta >
```

Figura 2.4: Verificación del tamaño en bytes de los resúmenes binarios.

Este experimento valida que el tamaño del hash es independiente del tamaño del mensaje de entrada; un archivo de 1 TB y uno de 1 byte generarán un hash SHA256 de exactamente 32 bytes en ambos casos.

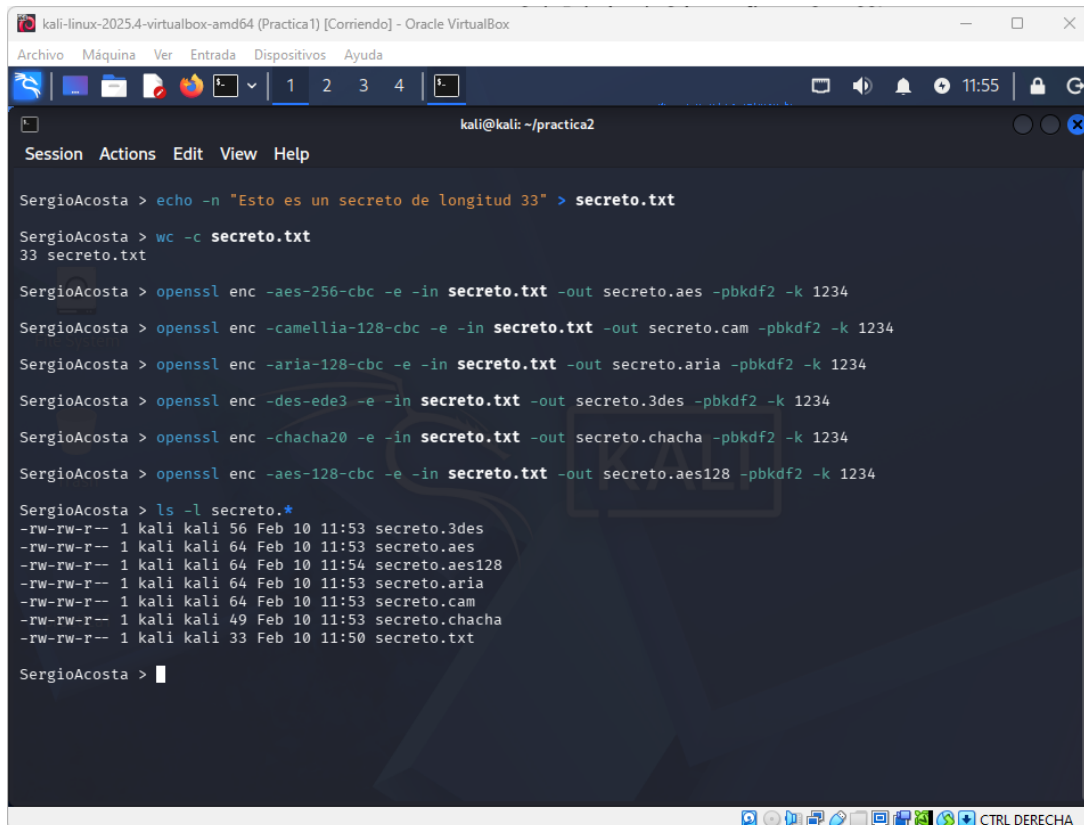
Capítulo 3

Cifrado Simétrico

En el cifrado simétrico, utilizamos la misma clave tanto para cifrar como para descifrar. En esta sección hemos evaluado diferentes algoritmos de bloque y de flujo.

3.1. Proceso de Cifrado y Algoritmos

Creamos un archivo secreto de exactamente 33 bytes de longitud: “Esto es un secreto de longitud 33”. Procedimos a cifrarlo utilizando `openssl enc` con cinco algoritmos distintos.



```
kali@kali: ~/practica2

Session Actions Edit View Help

SergioAcosta > echo -n "Esto es un secreto de longitud 33" > secreto.txt
SergioAcosta > wc -c secreto.txt
33 secreto.txt

SergioAcosta > openssl enc -aes-256-cbc -e -in secreto.txt -out secreto.aes -pbkdf2 -k 1234
SergioAcosta > openssl enc -camellia-128-cbc -e -in secreto.txt -out secreto.cam -pbkdf2 -k 1234
SergioAcosta > openssl enc -aria-128-cbc -e -in secreto.txt -out secreto.aria -pbkdf2 -k 1234
SergioAcosta > openssl enc -des-ede3 -e -in secreto.txt -out secreto.3des -pbkdf2 -k 1234
SergioAcosta > openssl enc -chacha20 -e -in secreto.txt -out secreto.chacha -pbkdf2 -k 1234
SergioAcosta > openssl enc -aes-128-cbc -e -in secreto.txt -out secreto.aes128 -pbkdf2 -k 1234

SergioAcosta > ls -l secreto.*
-rw-rw-r-- 1 kali kali 56 Feb 10 11:53 secreto.3des
-rw-rw-r-- 1 kali kali 64 Feb 10 11:53 secreto.aes
-rw-rw-r-- 1 kali kali 64 Feb 10 11:54 secreto.aes128
-rw-rw-r-- 1 kali kali 64 Feb 10 11:53 secreto.aria
-rw-rw-r-- 1 kali kali 64 Feb 10 11:53 secreto.cam
-rw-rw-r-- 1 kali kali 49 Feb 10 11:53 secreto.chacha
-rw-rw-r-- 1 kali kali 33 Feb 10 11:50 secreto.txt

SergioAcosta > 
```

Figura 3.1: Cifrado del archivo secreto con diversos algoritmos.

Los algoritmos seleccionados ilustran la evolución de la criptografía:

1. **AES-256-CBC:** El estándar actual (Advanced Encryption Standard). Utiliza bloques de 128 bits y una clave de 256 bits en modo Cipher Block Chaining (CBC).
2. **Camellia y Aria:** Algoritmos robustos desarrollados en Japón y Corea del Sur respectivamente, soportados por TLS como alternativas a AES.
3. **Triple DES (des-ede3):** Aplica el algoritmo DES tres veces para aumentar la seguridad, aunque es lento y computacionalmente costoso.
4. **ChaCha20:** Un cifrado de flujo (*stream cipher*) moderno, diseñado por Daniel J. Bernstein. Es extremadamente rápido y seguro, ideal para dispositivos móviles, y no utiliza bloques ni relleno.

3.2. Análisis del Tamaño del Cifrado (Padding)

Uno de los hallazgos más interesantes de la práctica es la discrepancia en el tamaño de los archivos cifrados resultantes.

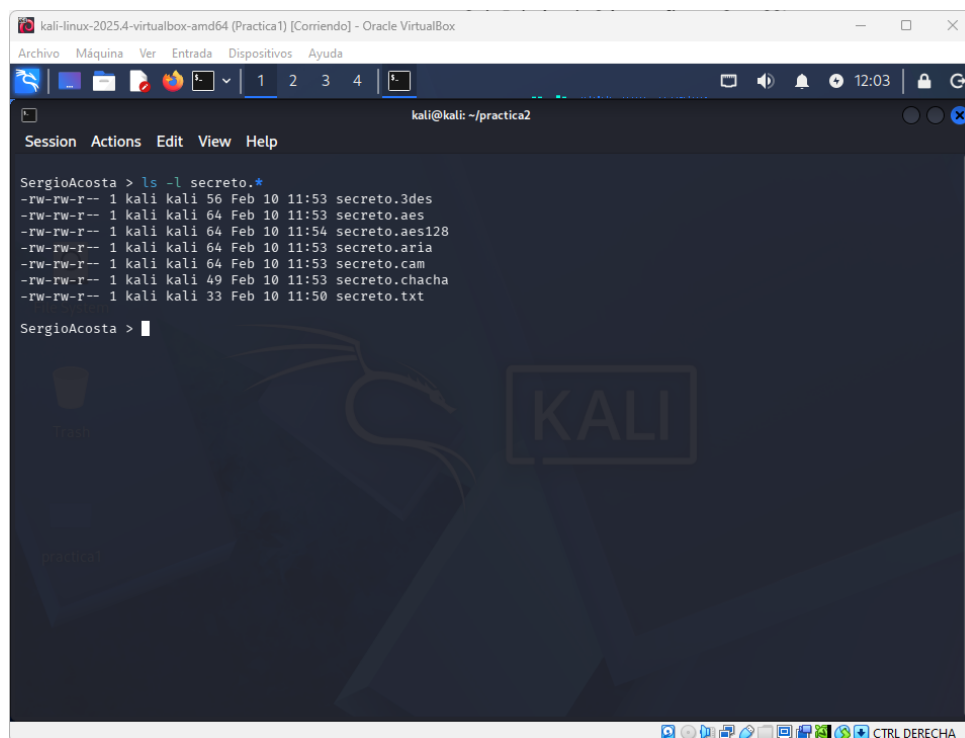


Figura 3.2: Comparativa de tamaños de los ficheros cifrados.

El archivo original pesa 33 bytes. Sin embargo, los resultados son:

- **secreto.chacha (49 bytes):** Al ser un cifrado de flujo, cifra bit a bit sin necesidad de bloques.

$$33(\text{datos}) + 16(\text{cabeceraSalt}) = 49\text{bytes}.$$

- **secreto.aes (64 bytes):** AES es un cifrado de bloque (16 bytes).

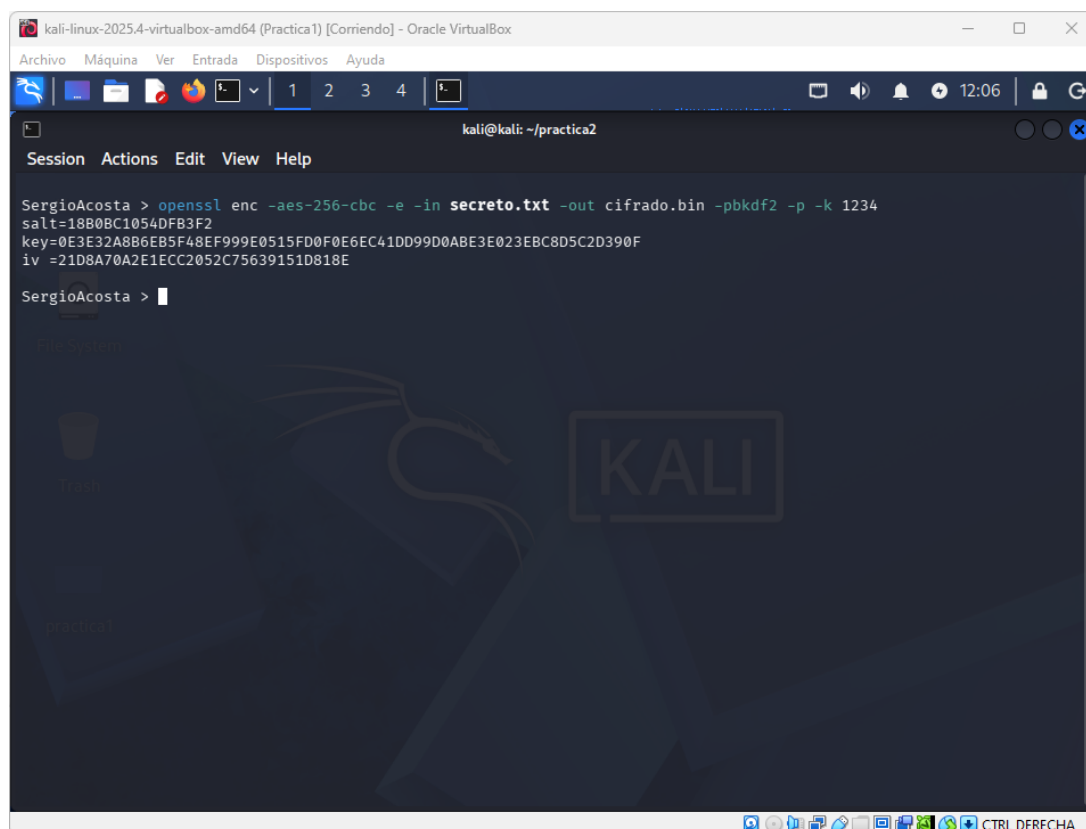
$$\text{Datos}(33) \rightarrow \text{Bloque1}(16) + \text{Bloque2}(16) + \text{Resto}(1).$$

Para completar el tercer bloque, se añade **Padding** (relleno) según el estándar PKCS#7. Se añaden 15 bytes de relleno para completar el bloque de 16.

$$48(\text{Cifrado}) + 16(\text{cabeceraSalt}) = 64\text{bytes}.$$

3.3. Derivación de Claves (KDF)

Cuando introducimos la contraseña “1234”, OpenSSL no la usa directamente como clave de cifrado. Utiliza una Función de Derivación de Clave (KDF), específicamente PBKDF2, para generar la **Key** y el **IV** (Vector de Inicialización).



```
kali@kali: ~/practica2
Session Actions Edit View Help

SergioAcosta > openssl enc -aes-256-cbc -e -in secreto.txt -out cifrado.bin -pbkdf2 -p -k 1234
salt=18B0BC1054DFB3F2
key=0E3E32A8B6EB5F48EF999E0515FD0F0E6EC41DD99D0ABE3E023EBC8D5C2D390F
iv =21D8A70A2E1ECC2052C756391510818E

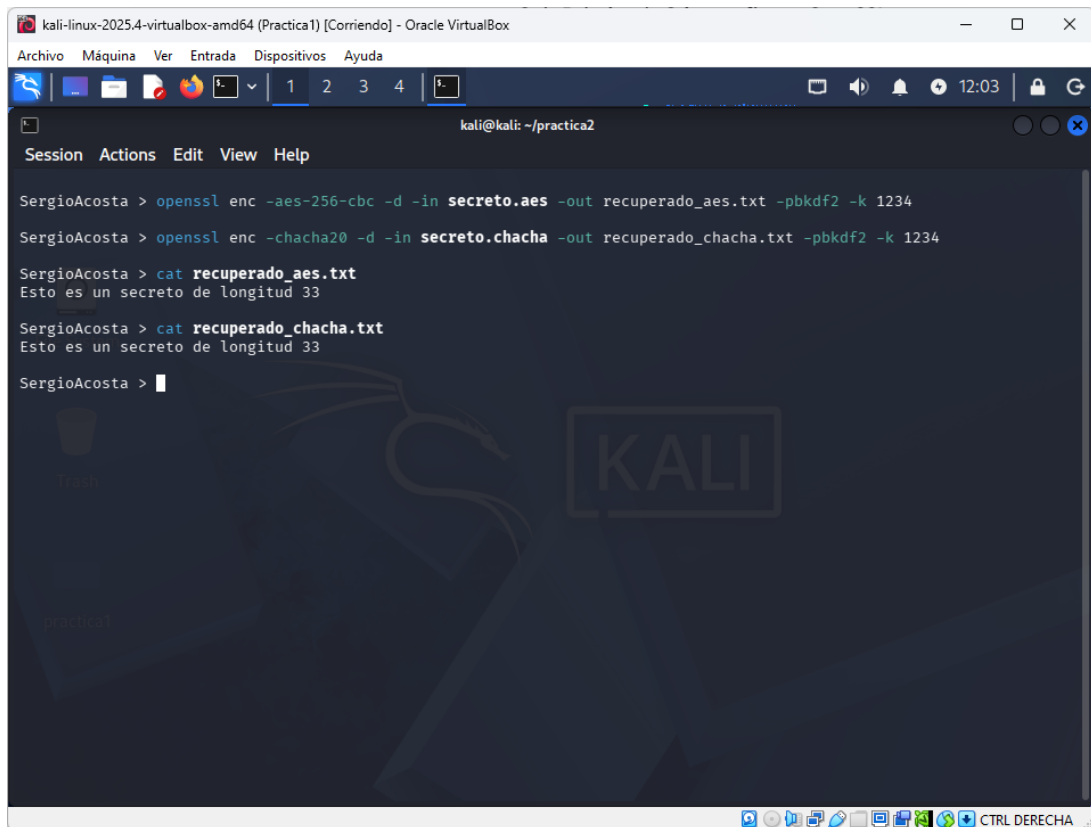
SergioAcosta > 
```

Figura 3.3: Visualización de parámetros Salt, Key e IV.

El uso de un **Salt** (valor aleatorio) garantiza que, si dos usuarios usan la misma contraseña “1234”, obtendrán claves de cifrado e IVs totalmente diferentes, protegiendo contra ataques de diccionario pre-calculados (Rainbow Tables).

3.4. Descifrado Manual (Avanzado)

Se verifica primero que el proceso es reversible de forma estándar.

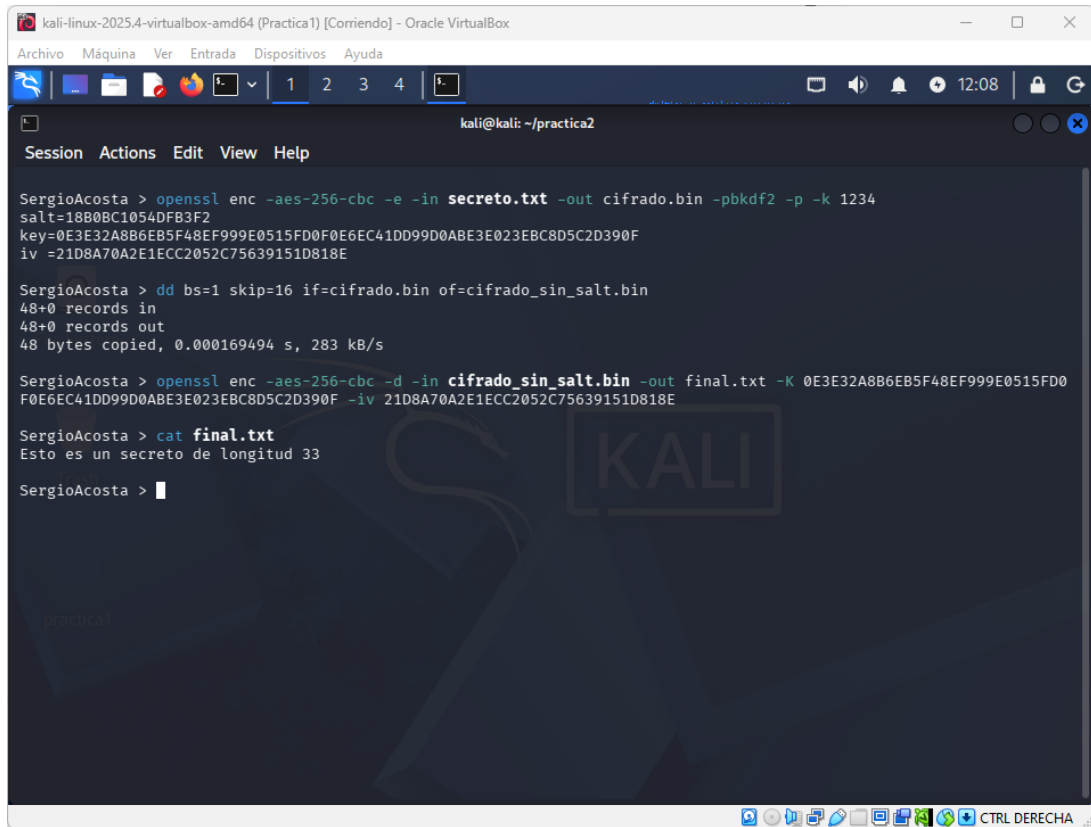


```
kali@kali: ~/practica2

SergioAcosta > openssl enc -aes-256-cbc -d -in secreto.aes -out recuperado_aes.txt -pbkdf2 -k 1234
SergioAcosta > openssl enc -chacha20 -d -in secreto.chacha -out recuperado_chacha.txt -pbkdf2 -k 1234
SergioAcosta > cat recuperado_aes.txt
Esto es un secreto de longitud 33
SergioAcosta > cat recuperado_chacha.txt
Esto es un secreto de longitud 33
SergioAcosta > 
```

Figura 3.4: Recuperación exitosa del texto plano.

Finalmente, realizamos un ejercicio avanzado de recuperación de datos sin usar la contraseña, sino los parámetros criptográficos directos.



```
kali@kali: ~/practica2

SergioAcosta > openssl enc -aes-256-cbc -e -in secreto.txt -out cifrado.bin -pbkdf2 -p -k 1234
salt=18B0BC1054DFB3F2
key=0E3E32A8B6EB5F48EF999E0515FD0F0E6EC41DD99D0ABE3E023EBC8D5C2D390F
iv =21D8A70A2E1ECC2052C75639151D818E

SergioAcosta > dd bs=1 skip=16 if=cifrado.bin of=cifrado_sin_salt.bin
48+0 records in
48+0 records out
48 bytes copied, 0.000169494 s, 283 kB/s

SergioAcosta > openssl enc -aes-256-cbc -d -in cifrado_sin_salt.bin -out final.txt -K 0E3E32A8B6EB5F48EF999E0515FD0F0E6EC41DD99D0ABE3E023EBC8D5C2D390F -iv 21D8A70A2E1ECC2052C75639151D818E

SergioAcosta > cat final.txt
Esto es un secreto de longitud 33

SergioAcosta > 
```

Figura 3.5: Descifrado manual manipulando cabeceras con dd.

El proceso consistió en:

1. **Eliminación de Cabecera:** OpenSSL añade una cabecera propietaria (`Salted__`) de 16 bytes al inicio del archivo. Para descifrar en crudo, debemos eliminarla. Usamos el comando `dd bs=1 skip=16` para copiar el archivo omitiendo esos primeros bytes.
2. **Descifrado Raw:** Ejecutamos OpenSSL indicando explícitamente la clave hexadecimal (`-K`) y el vector de inicialización (`-iv`). Esto descifra el contenido binario directamente, recuperando el texto original con éxito.

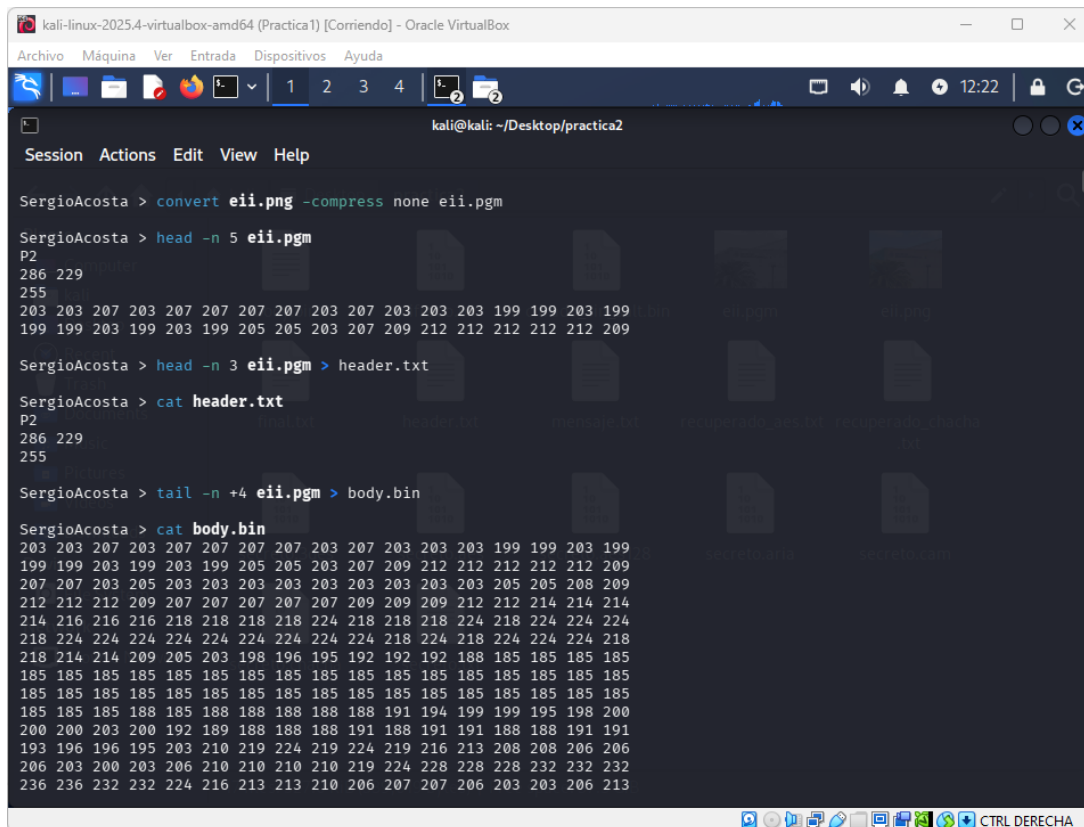
Capítulo 4

Análisis de Vulnerabilidad: Modo ECB

El modo *Electronic Codebook* (ECB) es la forma más simple de aplicar un cifrado de bloque, donde cada bloque de texto plano se cifra de forma independiente con la misma clave.

4.1. Manipulación de Imágenes (Netpbm)

Introducción a la manipulación de cabeceras de imagen para experimentos criptográficos.



```
kali@kali: ~/Desktop/practica2
SergioAcosta > convert eii.png -compress none eii.pgm

SergioAcosta > head -n 5 eii.pgm
P2
286 229
255
203 203 207 203 207 207 207 207 203 207 203 203 203 199 199 203 199
199 199 203 199 203 199 205 205 203 207 209 212 212 212 212 212 209

SergioAcosta > head -n 3 eii.pgm > header.txt

SergioAcosta > cat header.txt
P2
286 229
255

SergioAcosta > tail -n +4 eii.pgm > body.bin

SergioAcosta > cat body.bin
203 203 207 203 207 207 207 207 203 207 203 203 203 199 199 203 199
199 199 203 199 203 199 205 205 203 207 209 212 212 212 212 212 209
207 207 203 205 203 203 203 203 203 203 203 203 205 205 208 209
212 212 212 209 207 207 207 207 207 209 209 209 212 212 214 214 214
214 216 216 216 218 218 218 218 224 218 218 218 224 218 224 224 224
218 224 224 224 224 224 224 224 224 224 218 224 218 224 224 218
218 214 214 209 205 203 198 196 195 192 192 192 188 185 185 185 185
185 185 185 185 185 185 185 185 185 185 185 185 185 185 185 185
185 185 185 185 185 185 185 185 185 185 185 185 185 185 185 185
185 185 188 185 188 188 188 188 188 191 194 199 199 195 198 200
200 200 203 200 192 189 188 188 188 188 191 188 191 191 188 188 191 191
193 196 196 195 203 210 219 224 219 224 219 216 213 208 208 206 206
206 203 200 203 206 210 210 210 210 219 224 228 228 228 232 232 232
236 236 232 232 224 216 213 213 210 206 207 207 206 203 203 206 213
```

Figura 4.1: Conversión y separación de cabecera/cuerpo.

Se usa `convert` para transformar una imagen PNG a formato PGM. Se analiza la cabecera con `head` y se separa del cuerpo binario para poder cifrarlo sin corromper los metadatos.

4.2. El Problema de los Patrones Visuales

Para demostrar por qué ECB es inseguro, utilizamos la imagen del pingüino Tux. Las imágenes contienen grandes áreas de color uniforme (patrones). En formato PGM (Portable GrayMap), estas áreas se traducen en bloques de bytes idénticos.

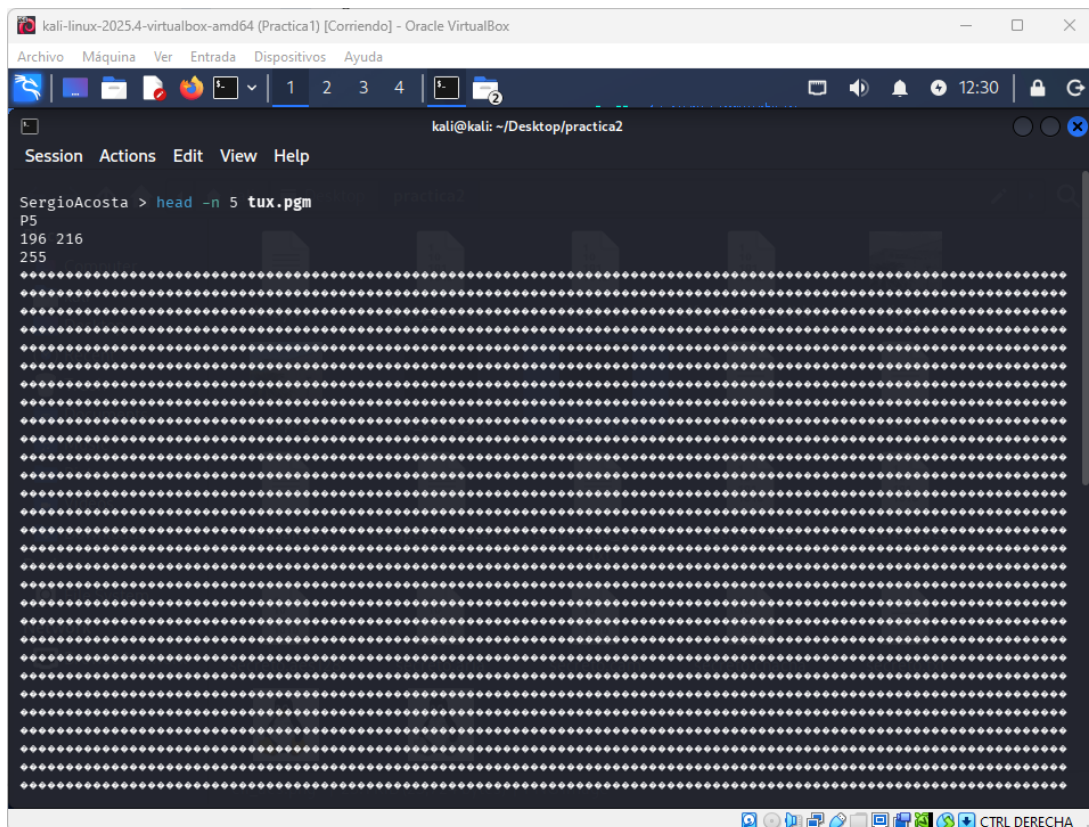


Figura 4.2: Imagen Tux en formato PGM (cabecera legible y cuerpo binario).

4.3. Ejecución del Ataque

El ataque consistió en separar la cabecera de la imagen (que define el tamaño y tipo) del cuerpo de datos (los píxeles).

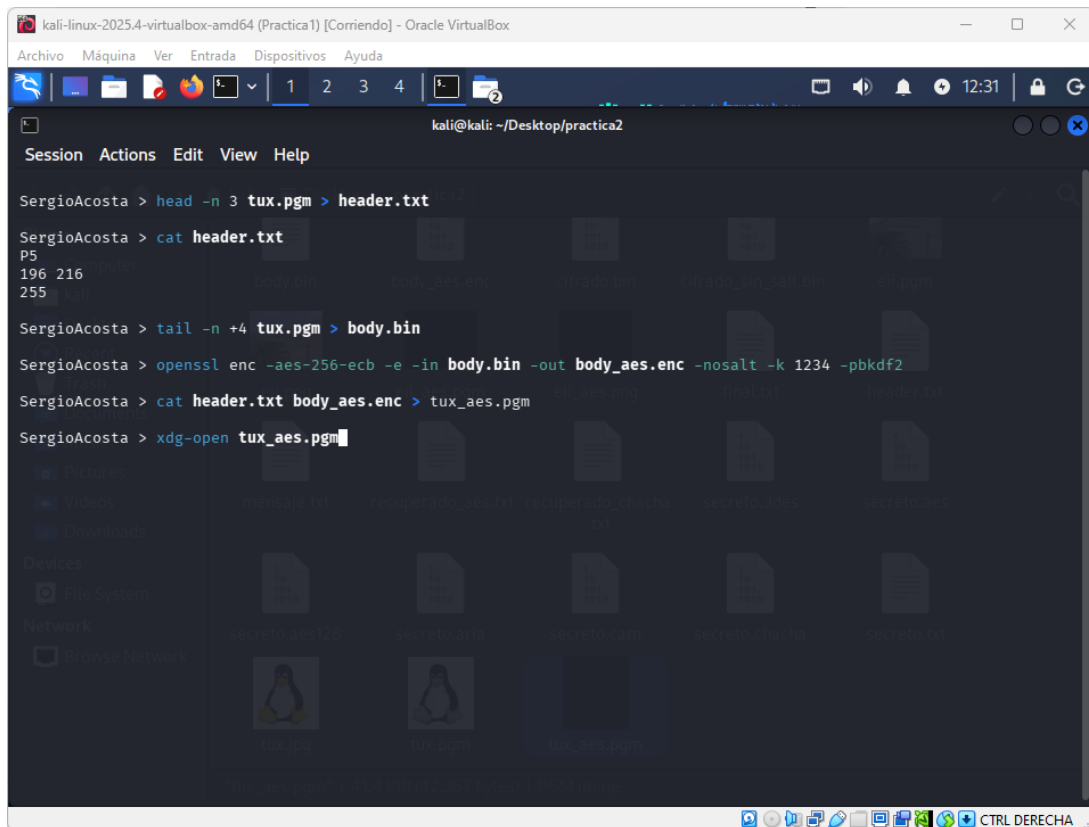


Figura 4.3: Cifrado del cuerpo de la imagen en modo ECB.

1. Separamos la cabecera (`head`) del cuerpo (`tail`).
2. Ciframos **solamente el cuerpo** utilizando AES-256 en modo **ECB**.
3. Volvimos a concatenar la cabecera original al cuerpo cifrado para que el visor de imágenes pudiera interpretarlo.

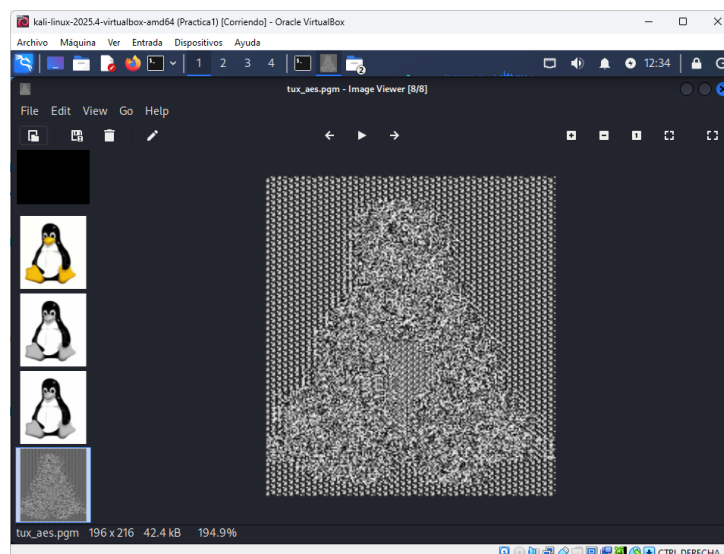


Figura 4.4: Resultado visual: La silueta del pingüino permanece visible.

Conclusión: Como se observa en la Figura 4.4, a pesar de usar un algoritmo robusto

(AES-256), el modo ECB preserva los patrones. Los bloques de píxeles blancos del fondo se cifraron siempre al mismo valor de salida, y los bloques negros del pingüino a otro valor, manteniendo los contornos de la imagen original.

Capítulo 5

Bibliografía

Referencias

- [1] The OpenSSL Project, “OpenSSL Manpages y Documentación Oficial,” [En línea]. Disponible: <https://www.openssl.org/docs/>. [Accedido: 13 de febrero de 2026].
- [2] NIST, “Security Guidelines for Storage Infrastructure,” SP 800-209, 2020.
- [3] B. Kaliski, “PKCS #5: Password-Based Cryptography Specification Version 2.0,” IETF RFC 2898, Sep. 2000.